Py

Python Learning Guide

WHAT • WHEN • HOW

32 Topics · Cheatsheets + Standard Library

Structured for Progressive Learning & Deep Understanding

01–23 Python Cheatsheets

24–32 Standard Library

Said Hussain Saim

Software Engineering Student

Daffodil International University, Bangladesh

Source: github.com/saim1sh/iPython_noteBook

Table of Contents

PART 1

Python Cheatsheets

Topics 01 — 23

TOPIC 01 Cheatsheet 01 · Variables, Types, Operators

Python Basics

W WHAT IS IT?

Python Basics covers the foundational building blocks of the language: variables, data types (int, float, str, bool, NoneType), arithmetic & comparison operators, type conversion, and the interactive print/input workflow. Every Python program starts here.

W WHEN TO USE?

Use these concepts from day one and in every program you ever write. Variable assignment, type checks with type(), and numeric/string operations appear in all domains: web backends, data science, automation, and AI.

H HOW TO USE?

Declare variables with simple assignment. Use f-strings for output. Cast between types with int(), float(), str(). Check types with isinstance(). Python is dynamically typed — no type declarations needed.

```
name = 'Saim' # str
age = 22 # int
gpa = 3.85 # float
is_student = True # bool
nothing = None # NoneType

print(f'Name: {name}, Age: {age}') # f-string
print(type(age)) # <class 'int'>
print(isinstance(age, int)) # True
x = int('42') # type casting
```

TOPIC 02 Cheatsheet 02 · Python's Power Tools

Built-in Functions

W WHAT IS IT?

Python ships with 70+ built-in functions available without any import. The most essential include: print(), input(), len(), range(), type(), isinstance(), int/float/str/bool conversions, min(), max(), sum(), abs(), round(), sorted(), reversed(), enumerate(), zip(), map(), filter(), open().

W WHEN TO USE?

Reach for built-ins before third-party libraries — they are faster, always available, and idiomatically Pythonic. Use enumerate() in every for-loop where you need an index. Use zip() to pair parallel lists. Use sorted() instead of list.sort() when you need a new list.

H HOW TO USE?

Call directly — no import needed. Chain them for concise code: sorted(set(items)) removes duplicates and sorts in one line. Use help(func) in REPL to see documentation instantly.

```
nums = [5, 3, 8, 1, 9, 2]
print(min(nums), max(nums), sum(nums)) # 1 9 28
print(sorted(nums)) # [1,2,3,5,8,9]

# enumerate: index + value
for i, v in enumerate(['a','b','c']):
print(i, v) # 0 a, 1 b, 2 c

# zip: pair two lists
names = ['Alice', 'Bob']
scores = [95, 87]
for n, s in zip(names, scores):
print(f'{n}: {s}')
```

TOPIC 03 Cheatsheet 03 · if/elif/else · for · while

Control Flow

W WHAT IS IT?

Control flow determines the order of execution. Python provides: if/elif/else for branching, for loops for iteration, while loops for condition-based repetition, and break/continue/pass for loop control. Match-case (Python 3.10+) adds structural pattern matching.

W WHEN TO USE?

Use if/elif/else whenever a decision must be made. Use for loops over any iterable (lists, strings, dicts, ranges). Use while loops when the number of iterations is unknown upfront — e.g., reading input until valid. Use break to exit early, continue to skip an iteration.

H HOW TO USE?

Python uses indentation (4 spaces) instead of braces. The colon (:) opens a block. Use range(start, stop, step) to generate number sequences. Combine with else on loops for 'completed without break' logic.

```
# if/elif/else
score = 78
if score >= 90: grade = 'A'
elif score >= 80: grade = 'B'
elif score >= 70: grade = 'C'
else: grade = 'F'

# for loop with range
for i in range(0, 10, 2): # 0,2,4,6,8
print(i, end=' ')

# while with break
n = 0
while True:
if n == 5: break
n += 1
```

TOPIC 04 Cheatsheet 04 · def · lambda · return

Functions

W WHAT IS IT?

Functions are reusable blocks of code defined with def. They accept parameters, perform operations, and optionally return values. Python supports default parameters, keyword arguments, lambda (anonymous) functions, and higher-order functions that accept/return other functions.

W WHEN TO USE?

Define a function whenever you find yourself copy-pasting the same code block. Functions improve readability, enable testing, and support the DRY principle. Lambda functions are ideal for short, single-expression operations passed to map(), filter(), sorted(), etc.

H HOW TO USE?

Use def name(params): followed by an indented body. Return values with return. Use default parameter values to make arguments optional. Add type hints for clarity. Docstrings go immediately after the def line.

```
# Basic function with default param
def greet(name: str, greeting: str = 'Hello') -> str:
'''Return a greeting string.'''
return f'{greeting}, {name}!'

print(greet('Saim')) # Hello, Saim!
print(greet('Bob', 'Hi')) # Hi, Bob!

# Lambda function
square = lambda x: x ** 2
nums = [1, 2, 3, 4]
squared = list(map(square, nums)) # [1,4,9,16]

# Higher-order function
evens = list(filter(lambda x: x % 2 == 0, nums))
```

TOPIC 05 Cheatsheet 05 · Sequences & Indexing

Lists and Tuples

W WHAT IS IT?

Lists are ordered, mutable sequences: [1, 2, 3]. Tuples are ordered, immutable sequences: (1, 2, 3). Both support indexing (list[0]), slicing (list[1:4]), and iteration. Lists have methods like append(), extend(), insert(), remove(), pop(), sort(), reverse(). Tuples are faster and hashable — usable as dict keys.

W WHEN TO USE?

Use lists when you need a mutable, ordered collection that will be modified (append, remove). Use tuples for fixed data (coordinates, RGB values, function return of multiple values) where immutability signals intent. Prefer tuples for performance-critical iteration.

H HOW TO USE?

Create with [] or list(). Slice with [start:stop:step]. Negative indices count from the end: list[-1] is the last element. Unpack tuples with a, b = (1, 2). Use * to capture the rest: first, *rest = [1,2,3,4].

```
fruits = ['apple', 'banana', 'cherry', 'date']
print(fruits[0]) # apple
print(fruits[-1]) # date
print(fruits[1:3]) # ['banana', 'cherry']
print(fruits[::-1]) # reverse

fruits.append('elderberry')
fruits.insert(1, 'avocado')
fruits.remove('banana')
popped = fruits.pop() # removes last

# Tuple unpacking
point = (10, 20, 30)
x, y, z = point
first, *rest = [1, 2, 3, 4, 5]
```

TOPIC 06 Cheatsheet 06 · Key-Value Mappings

Dictionaries

W WHAT IS IT?

Dictionaries are unordered (Python 3.7+ insertion-ordered), mutable collections of key-value pairs: {'key': value}. Keys must be hashable (strings, numbers, tuples). They provide O(1) average lookup. Key methods: get(), keys(), values(), items(), update(), pop(), setdefault().

W WHEN TO USE?

Use dicts whenever you need to associate keys with values — user profiles, config settings, caches, frequency counts, lookup tables. Prefer dict.get(key, default) over dict[key] to avoid KeyError. Use collections.defaultdict for auto-initializing values.

H HOW TO USE?

Create with {} or dict(). Iterate with .items() for key-value pairs. Merge dicts with {**d1, **d2} or d1 | d2 (Python 3.9+). Use dict comprehensions for transformation: {k: v*2 for k, v in d.items()}.

```
student = {'name': 'Saim', 'age': 22, 'gpa': 3.85}

# Access
print(student['name']) # Saim
print(student.get('dept', 'SWE')) # SWE (default)

# Modify
student['age'] = 23
student.update({'dept': 'SWE', 'year': 4})

# Iterate
for key, val in student.items():
print(f'{key}: {val}')

# Dict comprehension
squares = {n: n**2 for n in range(1, 6)}
```

TOPIC 07 Cheatsheet 07 · Unique Collections & Set Math

Sets

W WHAT IS IT?

A set is an unordered collection of unique, hashable elements: {1, 2, 3}. Sets automatically eliminate duplicates and support mathematical set operations: union (|), intersection (&), difference (-), symmetric difference (^). Useful methods: add(), remove(), discard(), pop(), clear().

W WHEN TO USE?

Use sets to remove duplicates from a list: list(set(my_list)). Use set operations to find common elements (intersection), all unique elements (union), or elements in one but not another (difference). Membership testing in sets is O(1) vs O(n) for lists.

H HOW TO USE?

Create with set() or {val1, val2}. Never use {} alone — that creates an empty dict. Use frozenset() for an immutable set (hashable, usable as dict key). Chain set operations: (a | b) - c.

```
a = {1, 2, 3, 4, 5}
b = {3, 4, 5, 6, 7}

print(a | b) # union: {1,2,3,4,5,6,7}
print(a & b) # intersection: {3,4,5}
print(a - b) # difference: {1,2}
print(a ^ b) # sym. diff: {1,2,6,7}

# Remove duplicates from a list
items = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(items)) # [1, 2, 3, 4]

# Fast membership test
allowed = {'admin', 'editor', 'viewer'}
print('admin' in allowed) # True — O(1)
```

TOPIC 08 Cheatsheet 08 · List · Dict · Set · Generator

Comprehensions

W WHAT IS IT?

Comprehensions are concise, readable syntax for creating collections from iterables. Python supports: list comprehensions [expr for x in iterable if cond], dict comprehensions {k: v ...}, set comprehensions {expr ...}, and generator expressions (expr ...) — lazy, memory-efficient versions.

W WHEN TO USE?

Use comprehensions instead of for-loops when building a new collection from an existing one with optional filtering. They are faster than equivalent loops and more Pythonic. Use generator expressions for large datasets to avoid loading everything into memory.

H HOW TO USE?

Structure: [expression for variable in iterable]. Add a condition: [expr for var in iterable if condition]. Nest for matrix operations. Wrap in () for a generator instead of []. Never use comprehensions for side effects — use regular loops for print() or file writes.

```
# List comprehension
squares = [x**2 for x in range(1, 6)] # [1,4,9,16,25]

# With filter
evens = [x for x in range(20) if x % 2 == 0]

# Dict comprehension
word_len = {w: len(w) for w in ['Python','is','cool']}

# Set comprehension
unique_chars = {c.lower() for c in 'Hello World'}

# Generator expression (memory-efficient)
total = sum(x**2 for x in range(1_000_000))

# Nested comprehension (matrix)
matrix = [[r*c for c in range(3)] for r in range(3)]
```

TOPIC 09 Cheatsheet 09 · Methods, Slicing, Encoding

Manipulating Strings

W WHAT IS IT?

Strings in Python are immutable sequences of Unicode characters. Python provides 40+ string methods: upper(), lower(), strip(), split(), join(), replace(), find(), index(), startswith(), endswith(), count(), encode(), decode(). Strings support slicing and all sequence operations.

W WHEN TO USE?

Use string methods for text parsing, cleaning, and transformation — extremely common in web development, data processing, and file I/O. Use split() to tokenize input, join() to combine lists into strings, strip() to clean whitespace from user input.

H HOW TO USE?

Call methods directly on string literals or variables. Chain them: ' Hello World '.strip().lower(). Use in operator for substring search. Strings are immutable — every method returns a new string.

```
s = ' Hello, World! '
print(s.strip()) # 'Hello, World!'
print(s.lower()) # ' hello, world! '
print(s.upper())
print(s.replace('World', 'Python'))
print(s.strip().split(', ')) # ['Hello', 'World!']

words = ['Python', 'is', 'powerful']
print(' '.join(words)) # 'Python is powerful'

# Slicing
text = 'abcdefgh'
print(text[2:6]) # cdef
print(text[::-1]) # hgfedcba (reverse)

print('world' in 'Hello world') # True
```

TOPIC 10 Cheatsheet 10 · f-strings, format(), %

String Formatting

W WHAT IS IT?

Python offers three main string formatting approaches: (1) f-strings (Python 3.6+): f'Hello {name}', the modern standard — fast and readable. (2) str.format(): 'Hello {}'.format(name) — compatible with older Python. (3) %-formatting: 'Hello %s' % name — legacy C-style, rarely used in new code.

W WHEN TO USE?

Prefer f-strings for all new code — they are the most readable and fastest. Use str.format() when building templates programmatically or for Python 2 compatibility. Use format spec mini-language for number/width formatting: f'{pi:.2f}', f'{n:>10}', f'{val:,}'.

H HOW TO USE?

Inside f-strings, use {variable} for simple insertion. Add format spec after colon: {value:.2f} for 2 decimals, {value:05d} for zero-padded int, {value:>20} for right-align. Use !r for repr(), !s for str(), !a for ascii().

```
name = 'Saim'
score = 98.567
big_num = 1234567

# f-string basics
print(f'Hello, {name}!')
print(f'Score: {score:.2f}') # 98.57
print(f'Big: {big_num:,}') # 1,234,567
print(f'{name:>20}') # right-align
print(f'{score:010.2f}') # 0000098.57

# str.format()
msg = 'Name: {}, Score: {:.1f}'.format(name, score)

# Multi-line f-string
report = (f'Student: {name}\n'
f'GPA: {score:.2f}')
```

TOPIC 11 Cheatsheet 11 · Pattern Matching with re

Regular Expressions

W WHAT IS IT?

Regular expressions (regex) are powerful patterns for matching, searching, and manipulating text. The re module provides: re.match() (beginning), re.search() (anywhere), re.findall() (all matches), re.sub() (replace), re.split() (split on pattern), re.compile() (pre-compile for reuse). Special chars: . * + ? ^ $ [] {} () | \

W WHEN TO USE?

Use regex when simple string methods (split, replace, find) are insufficient — validating emails/phone numbers, extracting structured data from unstructured text, log parsing, HTML/CSV parsing (with caution), and find-replace with complex patterns. Avoid regex for simple substring checks — use 'in' or str.find() instead.

H HOW TO USE?

Import re. Use raw strings r'pattern' to avoid backslash conflicts. Compile patterns with re.compile() if reused. Use groups () to capture parts: match.group(1). Add flags: re.IGNORECASE, re.MULTILINE.

```
import re

# Validate email
pattern = r'^[\w.-]+@[\w.-]+\.\w{2,}$'
email = 'saim@diu.edu.bd'
print(bool(re.match(pattern, email))) # True

# Extract all numbers from text
text = 'Order 123 costs 456.78 BDT'
nums = re.findall(r'\d+\.?\d*', text) # ['123','456.78']

# Replace with sub
clean = re.sub(r'\s+', ' ', 'too many spaces')

# Groups: extract date parts
m = re.search(r'(\d{4})-(\d{2})-(\d{2})', '2025-06-15')
year, month, day = m.group(1), m.group(2), m.group(3)
```

TOPIC 12 Cheatsheet 12 · pathlib & os.path

File & Directory Paths

W WHAT IS IT?

Python provides two ways to handle filesystem paths: the modern pathlib.Path (object-oriented, Python 3.4+) and the legacy os.path module. pathlib.Path objects support joining with /, existence checks (.exists(), .is_file(), .is_dir()), creation (.mkdir()), listing (.iterdir()), and pattern matching (.glob()).

W WHEN TO USE?

Always prefer pathlib over os.path for new code — it is cleaner, cross-platform, and more expressive. Use it whenever your program must navigate the filesystem: reading config files, processing directories of uploads, generating output paths, or building CLI tools.

H HOW TO USE?

from pathlib import Path. Join paths with /: Path('data') / 'output' / 'file.csv'. Get parts with .name, .stem, .suffix, .parent. Use Path.home() and Path.cwd() for home/current directory. Use glob('**/*.py') for recursive search.

```
from pathlib import Path

# Build paths cross-platform
base = Path('data')
output = base / 'results' / 'report.csv'

# Inspect
print(output.parent) # data/results
print(output.stem) # report
print(output.suffix) # .csv

# Create directory
output.parent.mkdir(parents=True, exist_ok=True)

# Glob: find all .py files recursively
for py_file in Path('.').glob('**/*.py'):
print(py_file)

p = Path.home() # /home/username
```

TOPIC 13 Cheatsheet 13 · open() · with · modes

Reading & Writing Files

W WHAT IS IT?

Python's built-in open() function handles file I/O. Modes: 'r' (read), 'w' (write, truncates), 'a' (append), 'b' (binary), 'x' (exclusive create). The context manager (with open() as f:) automatically closes files even if an exception occurs. Text files use readline(), readlines(), writelines(). Binary files use read(n) and write(bytes).

W WHEN TO USE?

Use file I/O for reading config files, processing data files (CSV, JSON, logs), generating reports, and saving application state. Always use with blocks — never call open() without a corresponding close(). Use 'a' mode for log files, 'w' for overwrite, 'r' for read-only.

H HOW TO USE?

with open('file.txt', 'r', encoding='utf-8') as f: is the standard pattern. Read all at once with f.read(). Iterate line by line with for line in f: (memory-efficient for large files). Write with f.write() or f.writelines().

```
# Write a file
with open('notes.txt', 'w', encoding='utf-8') as f:
f.write('Line 1\n')
f.writelines(['Line 2\n', 'Line 3\n'])

# Read all at once
with open('notes.txt', 'r', encoding='utf-8') as f:
content = f.read()

# Read line by line (memory-efficient)
with open('large_file.log') as f:
for line in f:
process(line.strip())

# Append to existing file
with open('log.txt', 'a') as f:
f.write('New entry\n')
```

TOPIC 14 Cheatsheet 14 · Serialization & Config Files

JSON and YAML

W WHAT IS IT?

JSON (JavaScript Object Notation) is the universal data interchange format. Python's json module converts: Python dict ↔ JSON string (json.dumps/loads) and Python dict ↔ JSON file (json.dump/load). YAML (Yet Another Markup Language) is a superset of JSON used for human-readable config files — requires PyYAML (pip install pyyaml).

W WHEN TO USE?

Use JSON for APIs, web data exchange, saving structured app state, and inter-process communication. Use YAML for configuration files (Django settings, Docker Compose, GitHub Actions, Kubernetes). JSON is strict but universal; YAML is more readable but has edge cases.

H HOW TO USE?

json.loads(string) → dict. json.dumps(dict) → string. json.load(file) → dict. json.dump(dict, file). Use indent=2 for pretty printing. For YAML: yaml.safe_load() for reading (safer than yaml.load()), yaml.dump() for writing.

```
import json

data = {'name': 'Saim', 'age': 22, 'skills': ['Python', 'JS']}

# Serialize to JSON string
json_str = json.dumps(data, indent=2)

# Deserialize from JSON string
parsed = json.loads(json_str)

# Write to file
with open('data.json', 'w') as f:
json.dump(data, f, indent=2)

# Read from file
with open('data.json') as f:
loaded = json.load(f)

# YAML (requires: pip install pyyaml)
import yaml
cfg = yaml.safe_load(open('config.yaml'))
```

TOPIC 15 Cheatsheet 15 · try/except/finally

Exception Handling

W WHAT IS IT?

Exception handling lets programs respond gracefully to errors instead of crashing. Python uses try/except/else/finally blocks. Common exceptions: ValueError, TypeError, KeyError, IndexError, FileNotFoundError, ZeroDivisionError, AttributeError, ImportError. You can raise custom exceptions by subclassing Exception. Use raise to re-raise or propagate errors.

W WHEN TO USE?

Wrap external calls (file I/O, network requests, user input, database queries) in try/except. Use specific exception types — never bare except:. Use finally for cleanup code that must run regardless (closing connections, files). Use else block for code that runs only when no exception occurred.

H HOW TO USE?

try: risky code. except ExceptionType as e: handling. else: no-error code. finally: always runs. Catch multiple with except (TypeError, ValueError). Create custom exceptions: class MyError(Exception): pass. Use raise MyError('message') to throw.

```
# Basic exception handling
try:
result = int(input('Enter number: '))
print(10 / result)
except ValueError:
print('Not a valid number!')
except ZeroDivisionError:
print('Cannot divide by zero!')
else:
print('Success!')
finally:
print('Always runs')

# Custom exception
class ValidationError(Exception): pass

def validate_age(age):
if age < 0: raise ValidationError('Age cannot be negative')
```

TOPIC 16 Cheatsheet 16 · pdb · print · logging

Debugging

W WHAT IS IT?

Debugging is the process of finding and fixing errors (bugs). Python provides: print() debugging (quick, dirty), the pdb module (Python Debugger — breakpoints, step-through), the logging module (structured, level-based output), and IDE debuggers (VS Code, PyCharm) that provide GUI breakpoints and variable inspection.

W WHEN TO USE?

Use print() for quick checks during development. Use logging for production code — it can be disabled without removing code. Use pdb or IDE debugger for complex bugs involving state, loops, or multiple function calls. Use assert for invariant checking during development.

H HOW TO USE?

Insert breakpoints with breakpoint() (Python 3.7+) or import pdb; pdb.set_trace(). pdb commands: n (next), s (step into), c (continue), q (quit), p (print variable). Set up logging: logging.basicConfig(level=logging.DEBUG). Use logging.debug(), .info(), .warning(), .error(), .critical().

```
import logging

# Setup logging
logging.basicConfig(
level=logging.DEBUG,
format='%(asctime)s %(levelname)s: %(message)s'
)

def divide(a, b):
logging.debug(f'divide({a}, {b})')
if b == 0:
logging.error('Division by zero attempted')
return None
return a / b

# Breakpoint (pdb)
def buggy():
x = [1, 2, 3]
breakpoint() # execution pauses here
return x[5] # IndexError to debug
```

TOPIC 17 Cheatsheet 17 · Variable Arguments

*args and **kwargs

W WHAT IS IT?

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. This enables functions to accept a variable number of arguments. The * and ** operators also work for unpacking in function calls: func(*my_list) passes list elements as positional args, func(**my_dict) passes dict as keyword args.

W WHEN TO USE?

Use *args when designing functions that should work with any number of inputs (like sum, print). Use **kwargs for configuration functions, decorators, and wrapper functions that must forward all arguments. Both are heavily used in frameworks (Django, Flask, FastAPI) for flexible API design.

H HOW TO USE?

def func(*args, **kwargs). Always keep order: positional → *args → keyword → **kwargs. Unpack a list with *: print(*[1, 2, 3]) → print(1, 2, 3). Merge dicts with **: {**dict1, **dict2}. Pass through with: def wrapper(*args, **kwargs): return func(*args, **kwargs).

```
# *args — variable positionals
def total(*args):
return sum(args)

print(total(1, 2, 3, 4)) # 10

# **kwargs — variable keywords
def profile(**kwargs):
for k, v in kwargs.items():
print(f'{k}: {v}')

profile(name='Saim', age=22, dept='SWE')

# Both together
def func(required, *args, **kwargs):
print(required, args, kwargs)

# Unpacking
nums = [1, 2, 3]
print(max(*nums)) # same as max(1,2,3)
```

TOPIC 18 Cheatsheet 18 · with · __enter__ · __exit__

Context Managers

W WHAT IS IT?

A context manager manages resource setup and teardown using the with statement. When you write with open() as f:, Python calls __enter__() before the block and __exit__() after — even if an exception occurs. You can create custom context managers with the contextlib.contextmanager decorator or by implementing __enter__ and __exit__ in a class.

W WHEN TO USE?

Use context managers for any resource that must be released after use: files, database connections, network sockets, locks (threading), temporary directories, and mock patches in testing. They eliminate resource leaks and guarantee cleanup without manual try/finally blocks.

H HOW TO USE?

The with statement is the primary interface. Use contextlib.contextmanager with a generator that yields once — code before yield is __enter__, code after is __exit__. For class-based: implement __enter__(self) returning self and __exit__(self, exc_type, exc_val, exc_tb) returning False (to propagate exceptions).

```
from contextlib import contextmanager
import time

# Custom context manager via decorator
@contextmanager
def timer(label):
start = time.time()
yield # block executes here
elapsed = time.time() - start
print(f'{label}: {elapsed:.4f}s')

with timer('my task'):
result = sum(range(1_000_000))

# Class-based context manager
class DBConnection:
def __enter__(self):
print('Connecting...')
return self
def __exit__(self, *args):
print('Closing connection')
```

TOPIC 19 Cheatsheet 19 · Classes · Inheritance · Polymorphism

OOP Basics

W WHAT IS IT?

Object-Oriented Programming (OOP) organizes code around objects (instances of classes). Key concepts: Class (blueprint), Object (instance), Attribute (data), Method (behavior), __init__ (constructor), self (instance reference). Pillars: Encapsulation (bundling data + methods), Inheritance (child extends parent), Polymorphism (same interface, different behavior), Abstraction (hide implementation).

W WHEN TO USE?

Use OOP when modeling real-world entities (User, Product, Order) or when building systems with related, interacting components. OOP shines in large codebases for maintainability and reuse. Prefer functions/modules for simple scripts. Frameworks (Django, SQLAlchemy) are built on OOP — understanding it is essential.

H HOW TO USE?

class ClassName: defines a class. def __init__(self, ...): is the constructor. Access attributes with self.attr. Call parent constructor with super().__init__(). Use @property for computed attributes. Use @classmethod and @staticmethod for alternative constructors and utility methods.

```
class Animal:
def __init__(self, name, sound):
self.name = name
self._sound = sound # _protected

def speak(self):
return f'{self.name} says {self._sound}'

@property
def info(self):
return f'Animal: {self.name}'

class Dog(Animal): # Inheritance
def __init__(self, name):
super().__init__(name, 'Woof')

def fetch(self, item):
return f'{self.name} fetches {item}!'

dog = Dog('Rex')
print(dog.speak()) # Rex says Woof
```

TOPIC 20 Cheatsheet 20 · @dataclass · field · frozen

Dataclasses

W WHAT IS IT?

Dataclasses (Python 3.7+, @dataclass decorator) auto-generate boilerplate methods: __init__, __repr__, __eq__. They are ideal for data containers that would otherwise require writing repetitive __init__ code. Options: frozen=True for immutable instances, order=True for comparison operators, field() for default_factory (mutable defaults like lists).

W WHEN TO USE?

Use dataclasses instead of regular classes when the primary purpose is storing data (DTOs, config objects, database models, API response objects). They replace verbose classes with manual __init__. Use frozen=True for value objects like Point, Color, or any object used as a dict key. Prefer over namedtuple for complex data with methods.

H HOW TO USE?

@dataclass decorator above class definition. Type annotations are required. Use field(default_factory=list) for mutable default values (never use [] directly). Use __post_init__ for validation after auto-generated __init__. from dataclasses import dataclass, field, asdict, astuple.

```
from dataclasses import dataclass, field

@dataclass
class Student:
name: str
student_id: int
gpa: float = 0.0
courses: list = field(default_factory=list)

def __post_init__(self):
if self.gpa < 0 or self.gpa > 4.0:
raise ValueError('GPA must be 0.0-4.0')

def enroll(self, course: str):
self.courses.append(course)

s = Student('Saim', 2021001, gpa=3.85)
s.enroll('Python')
print(s) # Auto __repr__
```

TOPIC 21 Cheatsheet 21 · setup.py · pyproject.toml

Package Setup

W WHAT IS IT?

A Python package is a directory with an __init__.py file. To distribute a package on PyPI, you need packaging metadata: pyproject.toml (modern standard) or setup.py/setup.cfg (legacy). pyproject.toml uses build backends like setuptools, flit, or poetry. It defines: name, version, description, dependencies, entry points, and classifiers.

W WHEN TO USE?

Use package setup when you want to: distribute your code via pip, organize a multi-module project with proper imports, define CLI entry points, or share code across multiple projects. Even for internal tools, a setup.py makes installation reproducible. Use pyproject.toml for all new projects.

H HOW TO USE?

Create pyproject.toml in project root. Use pip install -e . for editable install during development. Structure: src/ layout puts package inside src/ directory to avoid import confusion. Use python -m build + twine upload to publish to PyPI.

```
# pyproject.toml (modern standard)
[build-system]
requires = ['setuptools>=64']
build-backend = 'setuptools.backends.legacy:build'

[project]
name = 'my-package'
version = '0.1.0'
description = 'My Python package'
requires-python = '>=3.10'
dependencies = ['requests>=2.28', 'click>=8.0']

[project.scripts]
my-tool = 'my_package.cli:main'

# Install in editable mode for development
# pip install -e .
```

TOPIC 22 Cheatsheet 22 · __name__ == '__main__'

Main (Top-level Script)

W WHAT IS IT?

When Python runs a file directly, __name__ is set to '__main__'. When the same file is imported as a module, __name__ is the module's name. The guard if __name__ == '__main__': ensures certain code only runs when the file is executed directly — not when imported. This is essential for scripts with both importable functions AND runnable behavior.

W WHEN TO USE?

Always use the __main__ guard in any script that defines reusable functions AND has executable code (print statements, function calls, setup code). Without it, importing the file would run all top-level code unintentionally. It also enables test imports — test files can import the module without side effects.

H HOW TO USE?

Place all executable/entry-point code inside if __name__ == '__main__': Define functions, classes, and constants at the module level (always safe). Use def main(): for the entry point, then call it inside the guard. This is the pattern used by all professional Python CLI tools.

```
# utils.py
def add(a, b):
return a + b

def multiply(a, b):
return a * b

def main():
print('Running as script')
print(add(3, 4)) # 7
print(multiply(3, 4)) # 12

if __name__ == '__main__':
main()

# When imported:
# from utils import add -> works, no side effects
# python utils.py -> runs main()
```

TOPIC 23 Cheatsheet 23 · venv · pip · isolation

Virtual Environments

W WHAT IS IT?

A virtual environment (venv) is an isolated Python installation with its own site-packages. It prevents dependency conflicts between projects. Tools: venv (built-in), virtualenv, conda, poetry, pipenv. Key files: pyvenv.cfg, bin/python, bin/pip, Lib/site-packages. requirements.txt captures all installed packages with pip freeze.

W WHEN TO USE?

Always use a virtual environment for every Python project. Without it, all packages install globally and projects can break each other (Project A needs Django 3.2, Project B needs Django 4.2). Activate venv before installing anything. Use requirements.txt for reproducible deployments.

H HOW TO USE?

Create: python -m venv .venv. Activate: source .venv/bin/activate (Linux/Mac) or .venv\Scripts\activate (Windows). Deactivate: deactivate. Install packages: pip install package. Save: pip freeze > requirements.txt. Restore: pip install -r requirements.txt.

```
# Create virtual environment
python -m venv .venv

# Activate (Linux/Mac)
source .venv/bin/activate

# Activate (Windows)
.venv\Scripts\activate

# Install packages
pip install flask sqlalchemy pytest

# Save dependencies
pip freeze > requirements.txt

# Restore in new environment
pip install -r requirements.txt

# Deactivate
deactivate
```

PART 2

Standard Library

Topics 24 — 32

TOPIC 24 Standard Library 01 · copy module

copy — Shallow & Deep Copy

W WHAT IS IT?

The copy module provides copy.copy() (shallow copy) and copy.deepcopy() (deep copy). A shallow copy creates a new object but references the same nested objects. A deep copy creates a completely independent duplicate — new object AND new nested objects. Assignment (b = a) creates no copy at all — just another reference.

W WHEN TO USE?

Use copy.copy() when the object has no nested mutable objects (flat lists, simple dicts). Use copy.deepcopy() when the object contains nested lists, dicts, or objects that must be fully independent. Critical in: cloning game state, duplicating config objects, and functional programming patterns.

H HOW TO USE?

import copy. copy.copy(obj) for shallow, copy.deepcopy(obj) for deep. Lists have a shallow copy shortcut: new_list = original[:] or list.copy(). Dicts: new_dict = original.copy() (shallow). For classes, define __copy__ and __deepcopy__ to control copying behavior.

```
import copy

original = [[1, 2], [3, 4]]

# Assignment — same object!
ref = original
ref[0].append(99) # modifies original too!

# Shallow copy — new list, same inner lists
shallow = copy.copy(original)
shallow[0].append(99) # still modifies original[0]!

# Deep copy — fully independent
deep = copy.deepcopy(original)
deep[0].append(99) # original unchanged

print(original) # [[1, 2], [3, 4]]
print(deep) # [[1, 2, 99], [3, 4]]
```

TOPIC 25 Standard Library 02 · datetime module

datetime — Dates & Times

W WHAT IS IT?

The datetime module handles dates, times, and time deltas. Key classes: datetime.date (year, month, day), datetime.time (hour, minute, second), datetime.datetime (date + time combined), datetime.timedelta (duration/difference). Supports: parsing strings to datetime (strptime), formatting datetime to strings (strftime), timezone-aware datetimes with pytz or zoneinfo.

W WHEN TO USE?

Use datetime whenever your application deals with scheduling, logging timestamps, calculating durations, comparing dates, or displaying formatted dates/times. Always store timestamps in UTC internally and convert to local timezone for display. Use timedelta for date arithmetic (adding days, computing deadlines).

H HOW TO USE?

from datetime import datetime, date, timedelta. Now: datetime.now(). Parse: datetime.strptime(s, fmt). Format: dt.strftime(fmt). Common formats: '%Y-%m-%d %H:%M:%S', '%d/%m/%Y'. Arithmetic: date + timedelta(days=30). Difference: (dt2 - dt1).days.

```
from datetime import datetime, date, timedelta

# Current datetime
now = datetime.now()
print(now.strftime('%Y-%m-%d %H:%M:%S'))

# Parse from string
dt = datetime.strptime('2025-06-15', '%Y-%m-%d')

# Date arithmetic
deadline = date.today() + timedelta(days=30)
print(f'Deadline: {deadline}')

# Time difference
start = datetime(2025, 1, 1)
end = datetime(2025, 6, 15)
diff = (end - start).days
print(f'{diff} days elapsed')
```

TOPIC 26 Standard Library 03 · itertools module

itertools — Iterator Toolkit

W WHAT IS IT?

itertools provides fast, memory-efficient tools for creating iterators. Infinite iterators: count(), cycle(), repeat(). Finite combiners: chain(), islice(), compress(), dropwhile(), takewhile(), groupby(). Combinatoric iterators: product(), permutations(), combinations(), combinations_with_replacement(). All return lazy iterators — memory-efficient.

W WHEN TO USE?

Use itertools for: combining multiple iterables without creating intermediate lists (chain), working with infinite sequences (count for IDs), grouping sorted data (groupby), generating all combinations/permutations for algorithmic problems, and sliding window operations (pairwise, zip with islice).

H HOW TO USE?

from itertools import chain, groupby, combinations, product, etc. Results are lazy iterators — wrap with list() to see all results at once. groupby() requires sorted input to group correctly. Use itertools.islice(iterator, n) to take only first n elements of an infinite iterator.

```
from itertools import chain, combinations, groupby, product

# chain: flatten iterables
all_items = list(chain([1,2], [3,4], [5]))
# [1, 2, 3, 4, 5]

# combinations: choose r from n
teams = list(combinations(['A','B','C','D'], 2))
# [('A','B'),('A','C'),('A','D'),('B','C')...]

# groupby: group sorted data
data = [('A', 1), ('A', 2), ('B', 3), ('B', 4)]
for key, group in groupby(data, key=lambda x: x[0]):
print(key, list(group))

# product: cartesian product
pairs = list(product('AB', repeat=2))
# [('A','A'),('A','B'),('B','A'),('B','B')]
```

TOPIC 27 Standard Library 04 · json module deep-dive

json — JSON Processing

W WHAT IS IT?

The json module (standard library) handles JSON encoding and decoding. It supports: serializing Python objects to JSON strings/files, deserializing JSON to Python objects, custom encoders (JSONEncoder subclass) for non-serializable types like datetime, Decimal, and custom classes. Type mapping: dict↔object, list↔array, str↔string, int/float↔number, bool↔true/false, None↔null.

W WHEN TO USE?

Use the json module for REST API responses, configuration files, data storage, and inter-service communication. When working with datetimes or Decimal values in JSON, write a custom encoder. For large JSON files, use ijson for streaming parsing to avoid loading the entire file into memory.

H HOW TO USE?

Four core functions: json.dumps(obj) → string, json.loads(s) → object, json.dump(obj, file) → write file, json.load(file) → read file. Key params: indent (pretty-print), sort_keys, ensure_ascii, default (for custom types). For custom encoder: class MyEncoder(json.JSONEncoder): def default(self, obj):...

```
import json
from datetime import datetime

# Custom encoder for datetime
class DateTimeEncoder(json.JSONEncoder):
def default(self, obj):
if isinstance(obj, datetime):
return obj.isoformat()
return super().default(obj)

data = {'event': 'exam', 'date': datetime(2025, 6, 15)}
json_str = json.dumps(data, cls=DateTimeEncoder, indent=2)

# Pretty-print with sort
cfg = {'z': 3, 'a': 1, 'm': 2}
print(json.dumps(cfg, sort_keys=True, indent=2))

# Streaming large files: use json.load() line by line
with open('data.json') as f:
records = json.load(f)
```

TOPIC 28 Standard Library 05 · os module

os — Operating System Interface

W WHAT IS IT?

The os module provides a portable interface to operating system functionality: environment variables (os.environ), process management (os.getpid(), os.system()), directory operations (os.mkdir(), os.listdir(), os.getcwd(), os.chdir()), file operations (os.rename(), os.remove()), and path operations (os.path.*). Note: prefer pathlib over os.path for path handling.

W WHEN TO USE?

Use os for: reading environment variables (os.environ.get('API_KEY')), running shell commands (os.system() or better: subprocess), checking if a path exists (os.path.exists()), walking directory trees (os.walk()). Use pathlib for path manipulation. Use subprocess for shell commands.

H HOW TO USE?

import os. Most common: os.environ.get('VAR', 'default') for env vars. os.walk(top) yields (dirpath, dirnames, filenames) tuples recursively. os.makedirs(path, exist_ok=True) creates nested directories. os.path.join() builds paths (but prefer pathlib.Path / for new code).

```
import os

# Environment variables
db_url = os.environ.get('DATABASE_URL', 'sqlite:///dev.db')
api_key = os.environ.get('API_KEY') # None if missing

# Current directory and listing
print(os.getcwd())
files = os.listdir('.')

# Walk directory tree
for root, dirs, files in os.walk('./project'):
for f in files:
full_path = os.path.join(root, f)
print(full_path)

# Create nested directories
os.makedirs('output/reports/2025', exist_ok=True)

# File operations
os.rename('old_name.txt', 'new_name.txt')
```

TOPIC 29 Standard Library 06 · pathlib module

pathlib — Modern Path Handling

W WHAT IS IT?

pathlib (Python 3.4+) provides Path objects for cross-platform filesystem path manipulation. Path objects are string-like but with rich methods: .exists(), .is_file(), .is_dir(), .mkdir(), .rmdir(), .unlink(), .rename(), .read_text(), .write_text(), .glob(), .rglob(), .iterdir(). The / operator joins paths elegantly.

W WHEN TO USE?

Prefer pathlib over os.path for all new code. Use it whenever your program touches the filesystem: reading/writing files, listing directories, finding files by pattern. Path.read_text() and Path.write_text() replace open() for simple file operations. Use rglob('*.py') to find files recursively.

H HOW TO USE?

from pathlib import Path. Common workflow: p = Path('data') / 'file.csv'. Check: if p.exists(). Read: p.read_text(). Write: p.write_text(content). List: list(p.iterdir()). Find: list(p.rglob('*.json')). p.stat().st_size gives file size. p.with_suffix('.txt') changes extension.

```
from pathlib import Path

p = Path('data') / 'reports' / '2025' / 'june.txt'

# Simple read/write
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text('June Report\nData here', encoding='utf-8')
content = p.read_text(encoding='utf-8')

# Properties
print(p.name) # june.txt
print(p.stem) # june
print(p.suffix) # .txt
print(p.parent) # data/reports/2025

# Find all JSON files recursively
for f in Path('.').rglob('*.json'):
print(f.name, f.stat().st_size, 'bytes')
```

TOPIC 30 Standard Library 07 · random module

random — Randomization

W WHAT IS IT?

The random module generates pseudo-random numbers (not cryptographically secure). Key functions: random.random() (float 0.0–1.0), random.randint(a, b) (int inclusive), random.choice(seq) (random element), random.choices(seq, weights, k) (weighted choices), random.shuffle(list) (in-place shuffle), random.sample(seq, k) (k unique choices). Use secrets module for security-sensitive randomness.

W WHEN TO USE?

Use random for: simulations, games, sampling data, generating test data, shuffling playlists, random delays, Monte Carlo methods. Never use random for security-sensitive operations (tokens, passwords, session IDs) — use the secrets module instead. Use random.seed(n) for reproducible results in testing.

H HOW TO USE?

import random. Set seed for reproducibility: random.seed(42). random.choices() with weights is useful for probability-weighted selection. For sampling without replacement: random.sample(population, k). For secure randomness: import secrets; secrets.token_hex(16).

```
import random, secrets

random.seed(42) # reproducible results

print(random.random()) # 0.0 to 1.0
print(random.randint(1, 100)) # int 1-100
print(random.choice(['a','b','c']))

# Weighted random selection
outcomes = ['win', 'lose', 'draw']
weights = [0.4, 0.4, 0.2]
result = random.choices(outcomes, weights, k=1)[0]

# Shuffle in place
deck = list(range(1, 53))
random.shuffle(deck)

# Secure token (for passwords/sessions)
token = secrets.token_hex(16) # 32 char hex string
```

TOPIC 31 Standard Library 08 · shelve module

shelve — Persistent Storage

W WHAT IS IT?

shelve provides a persistent dictionary-backed by a database file. Unlike JSON, shelve can store any picklable Python object as values — including instances of custom classes, nested dicts, numpy arrays. It uses dbm internally and provides dict-like access: shelf['key'] = value. Not suitable for concurrent access or production databases.

W WHEN TO USE?

Use shelve for: simple persistent caches, storing program state between runs, quick prototypes needing persistence without setting up SQLite/PostgreSQL, small personal tools. Avoid shelve for: multi-user apps, concurrent access, large datasets, or when you need queryability (use SQLite or PostgreSQL instead).

H HOW TO USE?

with shelve.open('filename') as shelf: creates/opens the database. Use like a dict: shelf['key'] = value, val = shelf['key']. flag='r' for read-only, 'n' for new (clears existing), 'c' for create-if-not-exists. Set writeback=True to automatically track changes to mutable values.

```
import shelve

# Store objects persistently
with shelve.open('myapp_data') as shelf:
shelf['user'] = {'name': 'Saim', 'score': 100}
shelf['settings'] = {'theme': 'dark', 'lang': 'en'}

# Retrieve later (even after program restart!)
with shelve.open('myapp_data') as shelf:
user = shelf['user']
print(user['name']) # Saim

# Update value
data = shelf['user']
data['score'] += 10
shelf['user'] = data # must reassign

# Check existence
print('settings' in shelf) # True
```

TOPIC 32 Standard Library 09 · zipfile module

zipfile — ZIP Archives

W WHAT IS IT?

The zipfile module reads and writes ZIP archives. ZipFile class supports: extracting files (extract(), extractall()), listing contents (namelist(), infolist()), reading without extracting (open()), writing/adding files (write(), writestr()), and compression (ZIP_DEFLATED, ZIP_BZIP2, ZIP_LZMA). Supports password-protected ZIPs (read-only with pwd= parameter).

W WHEN TO USE?

Use zipfile for: packaging deployment files, creating archives for download, reading ZIP-based formats (XLSX, DOCX are ZIP files internally), compressing log files for storage, and batch file transfer. For TAR archives (Linux): use tarfile module. For simple in-memory compression: use gzip or zlib.

H HOW TO USE?

with zipfile.ZipFile('archive.zip', 'r') as zf: for reading. Use 'w' to create, 'a' to append to existing. zf.extractall('output/') extracts everything. zf.extract('file.txt', 'dest/') extracts one. zf.write('local_file.txt', arcname='inner_name.txt') adds with custom name.

```
import zipfile
from pathlib import Path

# Create a ZIP archive
with zipfile.ZipFile('project.zip', 'w', zipfile.ZIP_DEFLATED) as zf:
for py_file in Path('.').glob('*.py'):
zf.write(py_file, arcname=py_file.name)

# List contents
with zipfile.ZipFile('project.zip', 'r') as zf:
for info in zf.infolist():
print(f'{info.filename}: {info.file_size} bytes')

# Extract all
with zipfile.ZipFile('project.zip') as zf:
zf.extractall('extracted/')

# Read a file inside ZIP without extracting
with zipfile.ZipFile('project.zip') as zf:
with zf.open('main.py') as f:
print(f.read().decode())
```